Compte rendu

Sahki Enzo • 4 novembre 2026

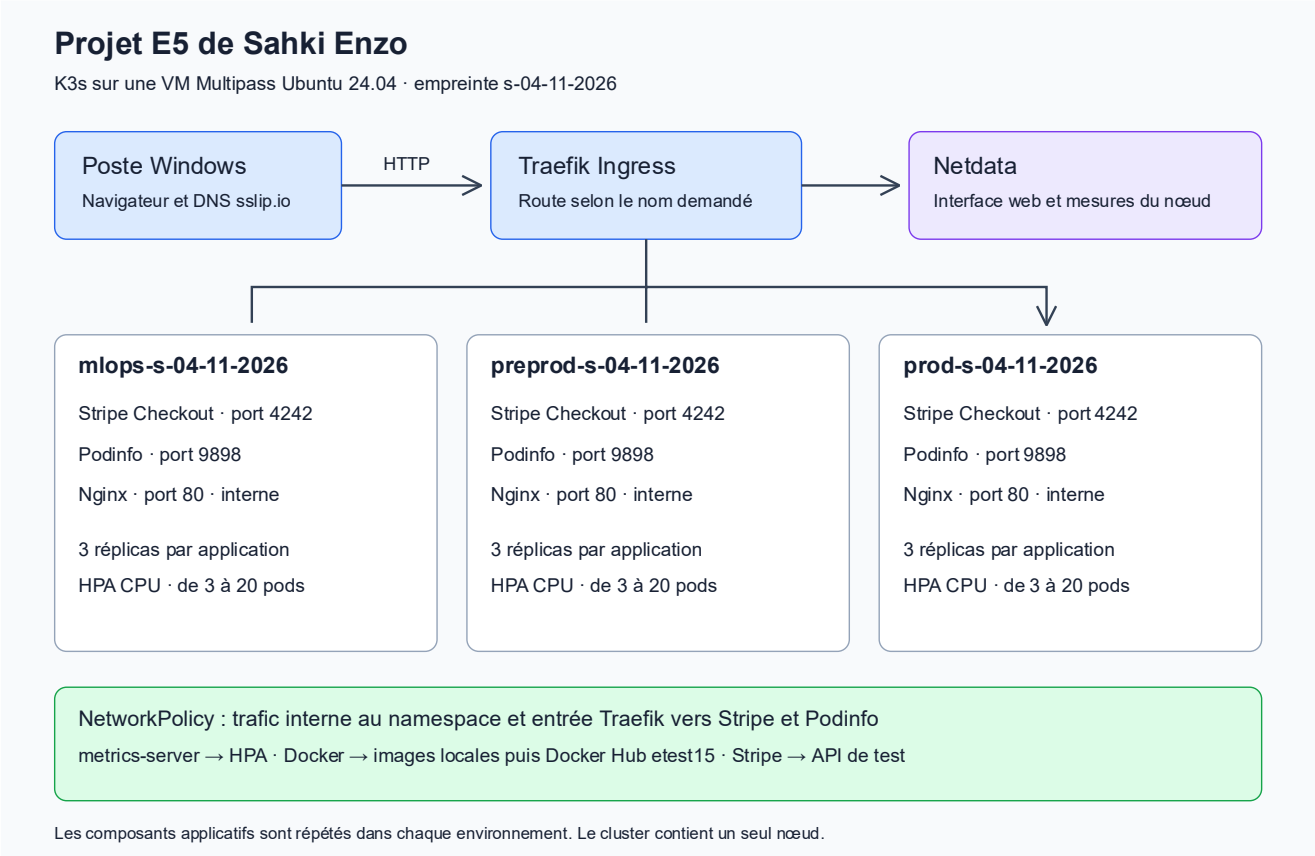
Empreinte des composants : s-04-11-2026

Le projet déploie trois applications existantes sur K3s, dans les environnements mlops, preprod et prod. Il comprend un paiement Stripe de test, des tests de charge et une supervision.

Applications et choix techniques

Stripe Checkout : exemple officiel avec Flask et HTML, repris sans modification. Podinfo : application de démonstration existante. Nginx : serveur web officiel utilisé en interne. Aucun code métier n'a été développé pour ces applications.

K3s est léger et simple à installer, mais une seule VM ne garantit pas la haute disponibilité. Les applications réutilisées permettent un MVP rapide ; elles ne constituent pas une boutique complète. Netdata permet de suivre les ressources dans une interface web.



Partie 1 Déploiement des applications

1 Mise en place de la VM et du dépôt Git

La VM Ubuntu possède 6 vCPU, 10 Go de RAM et 35 Go de disque. Les noms portent l'empreinte demandée et les objets applicatifs le label nom: sahki.

Dans PowerShell :

```
multipass list
multipass shell e5-s-04-11-2026
```

Dans Ubuntu, se placer dans le projet. Toutes les commandes suivantes sont exécutées dans cette VM.

```
cd /home/ubuntu/projet-e5-s-04-11-2026
sudo k3s kubectl get nodes -o wide
git log --oneline
```

```
ubuntu@e5-s-04-11-2026:~$ cd /home/ubuntu/projet-e5-s-04-11-2026
ubuntu@e5-s-04-11-2026:~/projet-e5-s-04-11-2026$ sudo k3s kubectl get nodes -o wide
NAME              STATUS   ROLES           AGE   VERSION        INTERNAL-IP      EXTERNAL-IP   OS-IMAGE             KERNEL-VERSION             CONTAINER-RUNTIME
e5-s-04-11-2026   Ready    control-plane   53m   v1.36.5+k3s1   172.26.141.177   <none>        Ubuntu 24.04.5 LTS   6.8.0-147-generic (amd64)   containerd://2.3.4-k3s1.36
ubuntu@e5-s-04-11-2026:~/projet-e5-s-04-11-2026$ git log --oneline
e98d273 (HEAD -> master) Garde les blocs de commandes entiers dans le rapport PDF
a7465aa Actualise la trace des commits pour le rendu
62063aa Valide le paiement Stripe et publie les trois images Docker Hub publiques
91505b2 Ajoute les preuves reelles de charge et le rapport individuel avec architecture
a312fd9 Corrige le delai de demarrage K3s et prepare les tests sans identifiants Stripe
2a4f6ed Ajout des applications existantes et des configurations K3s des trois environnements
ubuntu@e5-s-04-11-2026:~/projet-e5-s-04-11-2026$
```

2 Préparation et déploiement

Docker construit les images. Docker Hub les héberge. K3s les exécute avec containerd. Les configurations sont enregistrées dans Git et ne contiennent pas de commentaires dièse.

Ordre des scripts de préparation déjà utilisés. Il n'est pas nécessaire de les relancer pour prendre les captures :

```
bash scripts/01-installer-s-04-11-2026.sh
bash scripts/02-images-s-04-11-2026.sh
bash scripts/03-publier-s-04-11-2026.sh
bash scripts/04-secrets-s-04-11-2026.sh
bash scripts/05-deployer-s-04-11-2026.sh
bash scripts/06-dockerhub-s-04-11-2026.sh
bash scripts/10-adresses-s-04-11-2026.sh
```

Les trois images publiques sont etest15/stripe-s-04-11-2026:1, etest15/podinfo-s-04-11-2026:1 et etest15/nginx-s-04-11-2026:1.

[CAPTURE 3 À INSÉRER — Docker Hub : les trois dépôts du compte etest15 avec leurs images.]

```
sudo k3s kubectl get deploy,pods -n mlops-s-04-11-2026
```

```
ubuntu@e5-s-04-11-2026:~/projet-e5-s-04-11-2026$ sudo k3s kubectl get deploy,pods -n mlops-s-04-11-2026
NAME                                  READY   UP-TO-DATE   AVAILABLE   AGE
deployment.apps/nginx-s-04-11-2026    3/3     3            3           55m
deployment.apps/podinfo-s-04-11-2026  3/3     3            3           55m
deployment.apps/stripe-s-04-11-2026   3/3     3            3           55m

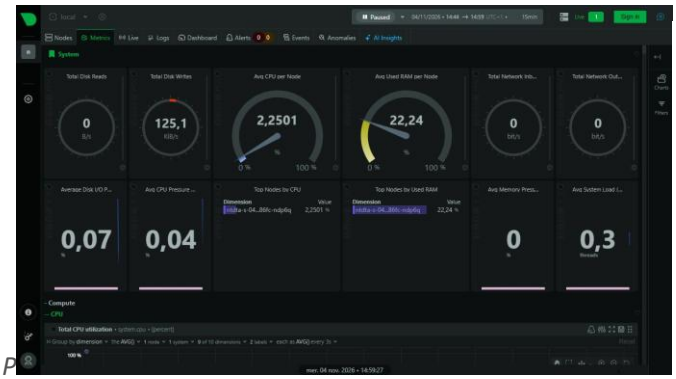
NAME                                      READY   STATUS    RESTARTS   AGE
pod/nginx-s-04-11-2026-788d9d895c-56qdb   1/1     Running   0          31m
pod/nginx-s-04-11-2026-788d9d895c-j2c7r   1/1     Running   0          31m
pod/nginx-s-04-11-2026-788d9d895c-n6bss   1/1     Running   0          31m
pod/podinfo-s-04-11-2026-64b46bbcf7-jlqcg 1/1     Running   0          31m
pod/podinfo-s-04-11-2026-64b46bbcf7-q52ts 1/1     Running   0          31m
pod/podinfo-s-04-11-2026-64b46bbcf7-wq2nh 1/1     Running   0          31m
pod/stripe-s-04-11-2026-65cc75549-4qpmq   1/1     Running   0          31m
pod/stripe-s-04-11-2026-65cc75549-cptn2   1/1     Running   0          31m
pod/stripe-s-04-11-2026-65cc75549-gkwpv   1/1     Running   0          31m
ubuntu@e5-s-04-11-2026:~/projet-e5-s-04-11-2026$
```

Le Deployment maintient les réplicas. Le Service distribue le trafic vers les pods. L'Ingress fournit l'accès HTTP extérieur au cluster.

3 Accès aux applications

```
sudo k3s kubectl get svc,ingress -n mlops-s-04-11-2026
cat logs/verification-adresses-s-04-11-2026.log
```

Stripe et Podinfo sont accessibles depuis Windows. Nginx reste interne. Les adresses sont indiquées dans ACCES-WINDOWS.txt. Le journal de vérification conserve les résultats des tests réseau.





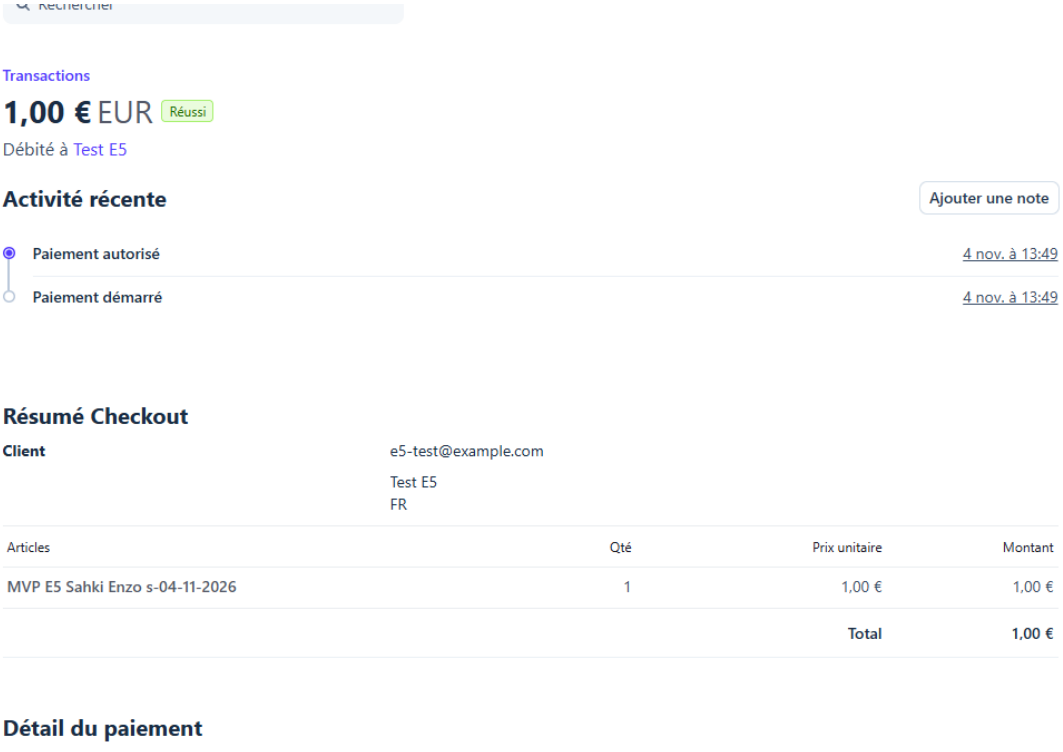
P

```
ubuntu@e5-s-04-11-2026:~/projet-e5-s-04-11-2026$ sudo k3s kubectl get svc,ingress -n mlops-s-04-11-2026
NAME                          TYPE        CLUSTER-IP      EXTERNAL-IP   PORT(S)    AGE
service/nginx-s-04-11-2026    ClusterIP   10.43.247.253   <none>        80/TCP     55m
service/podinfo-s-04-11-2026  ClusterIP   10.43.24.73     <none>        9898/TCP   55m
service/stripe-s-04-11-2026   ClusterIP   10.43.73.23     <none>        4242/TCP   55m

NAME                                                CLASS     HOSTS                                                                                                                                ADDRESS          PORTS   AGE
ingress.networking.k8s.io/applications-s-04-11-2026   traefik   stripe-mlops-s-04-11-2026.172-26-141-177.sslip.io,podinfo-mlops-s-04-11-2026.172-26-141-177.sslip.io   172.26.141.177   80      55m
ubuntu@e5-s-04-11-2026:~/projet-e5-s-04-11-2026$ cat logs/verification-adresses-s-04-11-2026.log
stripe-mlops-s-04-11-2026.172-26-141-177.sslip.io 200
podinfo-mlops-s-04-11-2026.172-26-141-177.sslip.io 200
nginx externe mlops 404
stripe-preprod-s-04-11-2026.172-26-141-177.sslip.io 200
podinfo-preprod-s-04-11-2026.172-26-141-177.sslip.io 200
nginx externe preprod 404
stripe-prod-s-04-11-2026.172-26-141-177.sslip.io 200
podinfo-prod-s-04-11-2026.172-26-141-177.sslip.io 200
nginx externe prod 404
pod/verification-s-04-11-2026 created
pod/verification-s-04-11-2026 condition met
INTERNE_OK
wget: can't connect to remote host (10.43.22.158): Connection refused
ISOLATION_OK
pod "verification-s-04-11-2026" deleted from mlops-s-04-11-2026 namespace
ubuntu@e5-s-04-11-2026:~/projet-e5-s-04-11-2026$
```

4 Paiement Stripe de test

Un paiement fictif de 1 € a été réalisé avec la carte de test 4242. Stripe confirme paid, complete et livemode=false. Le paiement réussi a aussi été vérifié dans le tableau de bord de test.



Partie 1 Ressources et autoscaling

5 Mesures CPU et mémoire

```
sudo k3s kubectl top pods -n mlops-s-04-11-2026
cat k8s/mlops/stripe.yaml
```

Au repos, la mémoire observée était d'environ 50 à 51 Mio pour Stripe, 12 Mio pour Podinfo et 5 Mio pour Nginx. Les ressources ont été dimensionnées avec une marge.

Stripe : réservation 50m CPU et 96Mi RAM, limites 500m et 256Mi. Podinfo : 25m et 32Mi, limites 300m et 128Mi. Nginx : 25m et 16Mi, limites 300m et 64Mi. 1000m correspond à un cœur CPU.



```
ubuntu@e5-s-04-11-2026:~/projet-e5-s-04-11-2026$ cat logs/verification-adresses-s-04-11-2026.log
stripe-mlops-s-04-11-2026.172-26-141-177.sslip.io 200
podinfo-mlops-s-04-11-2026.172-26-141-177.sslip.io 200
nginx externe mlops 404
stripe-preprod-s-04-11-2026.172-26-141-177.sslip.io 200
podinfo-preprod-s-04-11-2026.172-26-141-177.sslip.io 200
nginx externe preprod 404
stripe-prod-s-04-11-2026.172-26-141-177.sslip.io 200
podinfo-prod-s-04-11-2026.172-26-141-177.sslip.io 200
nginx externe prod 404
pod/verification-s-04-11-2026 created
pod/verification-s-04-11-2026 condition met
INTERNE_OK
wget: can't connect to remote host (10.43.22.158): Connection refused
ISOLATION_OK
pod "verification-s-04-11-2026" deleted from mlops-s-04-11-2026 namespace
ubuntu@e5-s-04-11-2026:~/projet-e5-s-04-11-2026$ sudo k3s kubectl top pods -n mlops-s-04-11-2026
cat k8s/mlops/stripe.yaml
NAME                                CPU(cores)   MEMORY(bytes)
nginx-s-04-11-2026-788d9d895c-56qdb     1m           6Mi
nginx-s-04-11-2026-788d9d895c-j2c7p     1m           6Mi
nginx-s-04-11-2026-788d9d895c-n6bss     1m           6Mi
podinfo-s-04-11-2026-64b46bbcf7-jlqcg   1m           13Mi
podinfo-s-04-11-2026-64b46bbcf7-q52ts   1m           13Mi
podinfo-s-04-11-2026-64b46bbcf7-wm2nh   1m           13Mi
stripe-s-04-11-2026-65cc75549-4qpmq     1m           57Mi
stripe-s-04-11-2026-65cc75549-cptn2     1m           52Mi
stripe-s-04-11-2026-65cc75549-gkmpv     1m           51Mi
apiVersion: apps/v1
kind: Deployment
metadata:
  name: stripe-s-04-11-2026
  namespace: mlops-s-04-11-2026
  labels:
    nom: sahki
    empreinte: s-04-11-2026
spec:
  replicas: 3
  selector:
    matchLabels:
      app: stripe-s-04-11-2026
  template:
    metadata:
      labels:
        app: stripe-s-04-11-2026
        nom: sahki
        empreinte: s-04-11-2026
    spec:
      containers:
      - name: stripe-s-04-11-2026
        image: etestl5/stripe-s-04-11-2026:1
        imagePullPolicy: IfNotPresent
        ports:
        - containerPort: 4242
        envFrom:
        - secretRef:
            name: stripe-s-04-11-2026
        env:
          - containerPort: 4242
          envFrom:
          - secretRef:
              name: stripe-s-04-11-2026
          env:
          - name: DOMAIN
            value: http://stripe-mlops-s-04-11-2026.172-26-141-177.sslip.io
          resources:
            requests:
              cpu: 50m
              memory: 96Mi
            limits:
              cpu: 500m
              memory: 256Mi
          readinessProbe:
            httpGet:
              path: /
              port: 4242
            initialDelaySeconds: 5
            periodSeconds: 10
          livenessProbe:
            httpGet:
              path: /
              port: 4242
            initialDelaySeconds: 20
            periodSeconds: 15
---
apiVersion: v1
kind: Service
metadata:
  name: stripe-s-04-11-2026
  namespace: mlops-s-04-11-2026
  labels:
    nom: sahki
    empreinte: s-04-11-2026
spec:
  type: ClusterIP
  selector:
    app: stripe-s-04-11-2026
  ports:
  - name: http-s-04-11-2026
    port: 4242
    targetPort: 4242
---
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata:
  name: stripe-s-04-11-2026
  namespace: mlops-s-04-11-2026
  labels:
    nom: sahki
    empreinte: s-04-11-2026
spec:
  scaleTargetRef:
    apiVersion: apps/v1
    kind: Deployment
    name: stripe-s-04-11-2026
  minReplicas: 3
  maxReplicas: 20
  metrics:
  - type: Resource
    resource:
      name: cpu
      target:
        type: Utilization
        averageUtilization: 60
  behavior:
    scaleDown:
      stabilizationWindowSeconds: 120
```

6 Test de charge et retour au repos

Le HPA utilise les mesures CPU de metrics-server. Sa cible est 60 % de la réservation CPU, avec 3 réplicas minimum et 20 maximum. La descente utilise une fenêtre de stabilisation de 120 secondes.

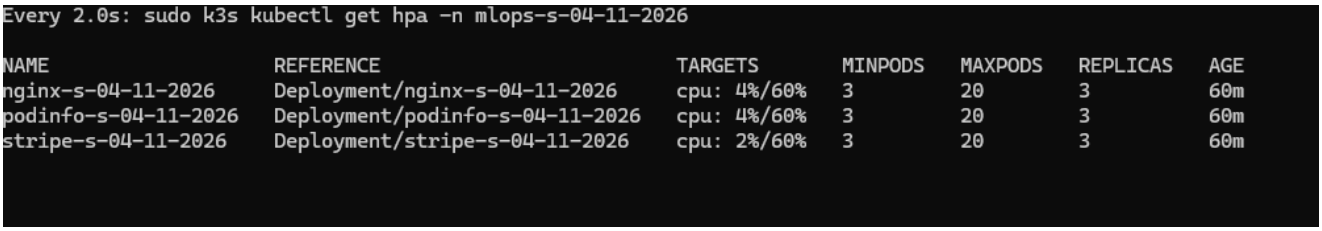
Pour de nouvelles captures, ouvrir deux terminaux Ubuntu dans le dossier du projet. Dans le premier, lancer les tests suivants un par un, en attendant la fin de chacun :

```
bash scripts/07-charge-s-04-11-2026.sh mlops stripe
bash scripts/07-charge-s-04-11-2026.sh mlops podinfo
bash scripts/07-charge-s-04-11-2026.sh mlops nginx
```

Dans le deuxième terminal :

```
watch -n 2 sudo k3s kubectl get hpa -n mlops-s-04-11-2026
```

Chaque script envoie une charge pendant 180 secondes et observe le cluster pendant environ six minutes. Il actualise ses fichiers de logs. Ctrl+C arrête uniquement la surveillance watch.



Lors des tests réalisés, les trois applications de mlops sont passées de 3 à 20 réplicas, puis sont revenues à 3. Les autres environnements sont restés au repos. Une nouvelle mesure doit être décrite selon les valeurs réellement observées.

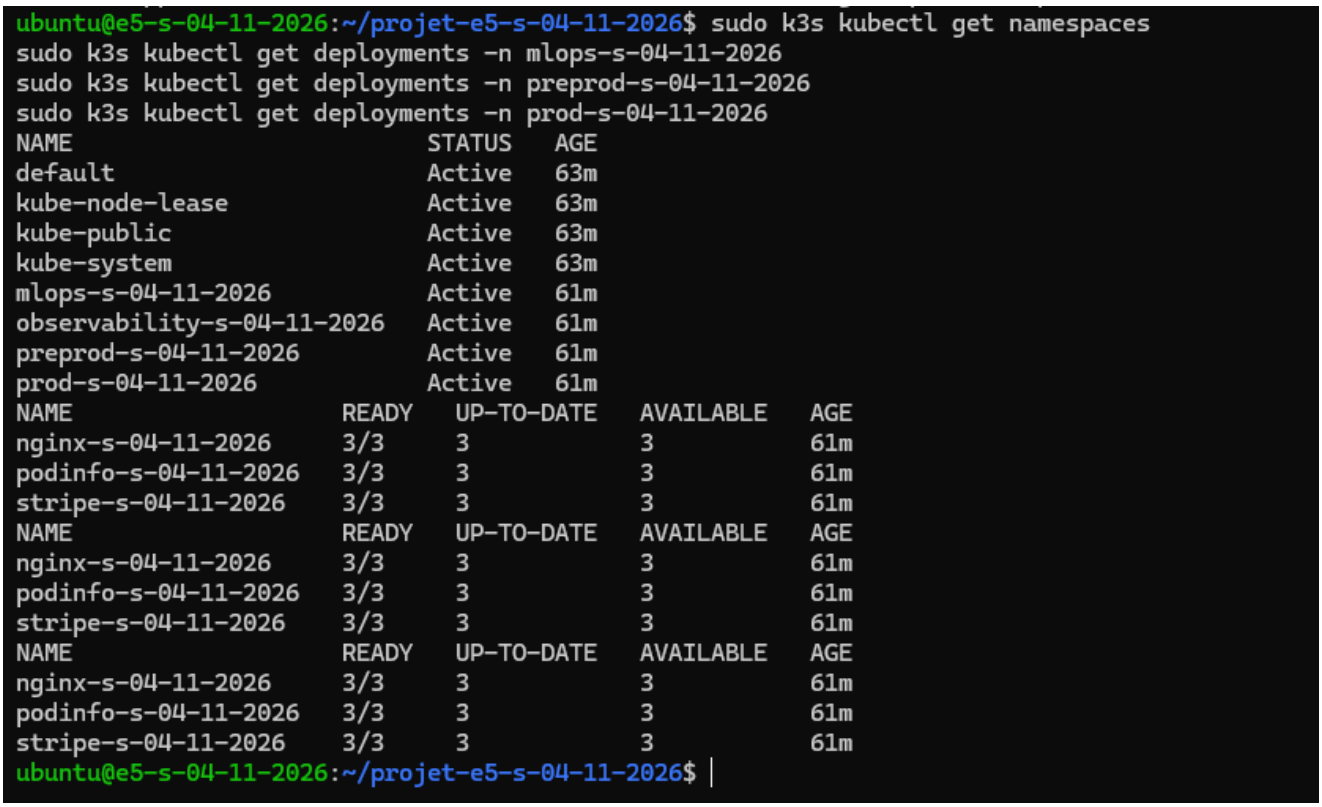
Le test charge la page d'accueil Stripe ; il ne crée pas des paiements en boucle. Les résultats sont conservés dans logs/autoscaling-mlops-*.log et logs/charge-mlops-*.log.

Partie 2 Environnements isolés

7 Même infrastructure dans les trois environnements

```
sudo k3s kubectl get namespaces
sudo k3s kubectl get deployments -n mlops-s-04-11-2026
sudo k3s kubectl get deployments -n preprod-s-04-11-2026
sudo k3s kubectl get deployments -n prod-s-04-11-2026
```

Les trois namespaces contiennent les mêmes applications et les mêmes réglages. Les noms d'environnement et les adresses changent. Cela représente neuf Deployments et 27 pods applicatifs au repos.



8 Isolation réseau

```
sudo k3s kubectl get networkpolicy -A
sudo k3s kubectl describe networkpolicy isolation-s-04-11-2026 -n mlops-s-04-11-2026
cat logs/verification-adresses-s-04-11-2026.log
```

Les NetworkPolicy autorisent le trafic du même namespace et l'entrée de Traefik vers Stripe et Podinfo. Le test confirme que mlops ne peut pas joindre Nginx dans prod. Un namespace seul ne suffit pas à bloquer le trafic.

```
ubuntu@e5-s-04-11-2026:~/projet-e5-s-04-11-2026$ watch -n 2 sudo k3s kubectl get hpa -n mlops-s-04-11-2026

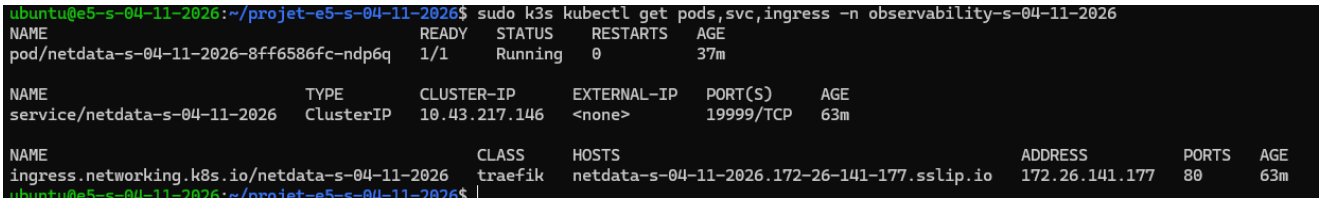
[1]+  Stopped                 watch -n 2 sudo k3s kubectl get hpa -n mlops-s-04-11-2026
ubuntu@e5-s-04-11-2026:~/projet-e5-s-04-11-2026$ sudo k3s kubectl get namespaces
sudo k3s kubectl get deployments -n mlops-s-04-11-2026
sudo k3s kubectl get deployments -n preprod-s-04-11-2026
sudo k3s kubectl get deployments -n prod-s-04-11-2026
NAME                          STATUS   AGE
default                       Active   63m
kube-node-lease               Active   63m
kube-public                   Active   63m
kube-system                   Active   63m
mlops-s-04-11-2026            Active   61m
observability-s-04-11-2026    Active   61m
preprod-s-04-11-2026          Active   61m
prod-s-04-11-2026             Active   61m
NAME                  READY   UP-TO-DATE   AVAILABLE   AGE
nginx-s-04-11-2026    3/3     3            3           61m
podinfo-s-04-11-2026  3/3     3            3           61m
stripe-s-04-11-2026   3/3     3            3           61m
NAME                  READY   UP-TO-DATE   AVAILABLE   AGE
nginx-s-04-11-2026    3/3     3            3           61m
podinfo-s-04-11-2026  3/3     3            3           61m
stripe-s-04-11-2026   3/3     3            3           61m
NAME                  READY   UP-TO-DATE   AVAILABLE   AGE
nginx-s-04-11-2026    3/3     3            3           61m
podinfo-s-04-11-2026  3/3     3            3           61m
stripe-s-04-11-2026   3/3     3            3           61m
ubuntu@e5-s-04-11-2026:~/projet-e5-s-04-11-2026$ sudo k3s kubectl get networkpolicy -A
sudo k3s kubectl describe networkpolicy isolation-s-04-11-2026 -n mlops-s-04-11-2026
cat logs/verification-adresses-s-04-11-2026.log
NAMESPACE             NAME                      POD-SELECTOR   AGE
mlops-s-04-11-2026    isolation-s-04-11-2026    <none>         62m
preprod-s-04-11-2026  isolation-s-04-11-2026    <none>         62m
prod-s-04-11-2026     isolation-s-04-11-2026    <none>         62m
Name:         isolation-s-04-11-2026
Namespace:    mlops-s-04-11-2026
Created on:   2026-11-04 14:32:06 +0100 CET
Labels:       empreinte=s-04-11-2026
              nom=sahki
Annotations:  <none>
Spec:
  PodSelector:     <none> (Allowing the specific traffic to all pods in this namespace)
  Allowing ingress traffic:
    To Port: <any> (traffic allowed to all ports)
    From:
      PodSelector: <none>
    ----------
    To Port: 4242/TCP
    To Port: 9898/TCP
    From:
      NamespaceSelector: kubernetes.io/metadata.name=kube-system
      PodSelector: app.kubernetes.io/name=traefik
  Not affecting egress traffic
  Policy Types: Ingress
stripe-mlops-s-04-11-2026.172-26-141-177.sslip.io 200
podinfo-mlops-s-04-11-2026.172-26-141-177.sslip.io 200
nginx externe mlops 404
stripe-preprod-s-04-11-2026.172-26-141-177.sslip.io 200
podinfo-preprod-s-04-11-2026.172-26-141-177.sslip.io 200
nginx externe preprod 404
stripe-prod-s-04-11-2026.172-26-141-177.sslip.io 200
podinfo-prod-s-04-11-2026.172-26-141-177.sslip.io 200
nginx externe prod 404
pod/verification-s-04-11-2026 created
pod/verification-s-04-11-2026 condition met
INTERNE_OK
wget: can't connect to remote host (10.43.22.158): Connection refused
```

Partie 3 Supervision

9 Interface Netdata

`sudo k3s kubectl get pods,svc,ingress -n observability-s-04-11-2026`

Netdata 2.12.2 affiche les mesures de la VM et des ressources visibles. Son interface permet notamment de suivre le CPU et la RAM. Sur l'accueil, choisir « Skip and use the dashboard anonymously ».



Adresse de la supervision :

http://netdata-s-04-11-2026.172-26-141-177.sslip.io/

Bilan et pièces jointes

Difficultés rencontrées

K3s : le premier contrôle arrivait avant l'enregistrement du nœud. Une attente a été ajoutée. Podinfo : ApacheBench comptait les variations de taille des réponses comme des erreurs ; l'option -l a corrigé ce point. Docker Hub : la connexion par code échouait ; la connexion interactive a permis la publication.

Résultats

Les trois applications fonctionnent dans les trois environnements. Les images sont publiques sur Docker Hub, le paiement Stripe de test a réussi et l'autoscaling a été vérifié sur mlops. Netdata est accessible depuis Windows. La VM unique reste une infrastructure de démonstration.

Journaux à joindre

Pour actualiser les preuves Kubernetes :

bash scripts/08-preuves-s-04-11-2026.sh

Joindre le dossier logs, les dossiers k8s et scripts, les Dockerfile et le dépôt Git. Le fichier commandes-agent précise les commandes exécutées par l'agent ; il ne représente pas un historique personnel fictif. Les captures complètent les journaux.

Pour enregistrer une nouvelle session personnelle, lancer cette commande avant les manipulations, puis saisir exit à la fin :

```
script -a logs/history-projet-SAHKI-ENZO.log
```

Le fichier .env et le dossier secrets doivent rester hors du rendu.

Sources des applications et des outils

Stripe Checkout : github.com/stripe-samples/checkout-one-time-payments
Podinfo : github.com/stefanprodan/podinfo
Nginx : hub.docker.com/_/nginx
K3s : docs.k3s.io
Kubernetes : kubernetes.io/docs
Netdata : learn.netdata.cloud

Les applications ont été réutilisées. Le travail du projet concerne leur conteneurisation, leur configuration et leur déploiement.